# Tokens, pairs, and first working programs

All examples in this section are new illustrations, not official contest samples. Each language block is a complete program. Inputs satisfy the stated format; lengths and sums fit the declared C++ types. Read the case framing before coding.

## 01 One integer + C++ fast-I/O setup

**C++17 - complete program**

```
#include <iostream>
using namespace std; int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long x;
    if (!(cin >> x)) return 0;
    cout << x << '\n';
}
```

WATCH: The two C++ setup lines belong before any I/O. After disabling synchronization, keep one I/O family; avoid mixing cin/cout with scanf/printf. Python bytes work with int().

Read one integer; print it unchanged.

IN: 42

OUT: 42

**PYTHON 3 - complete program**

```
import sys
s = sys.stdin.buffer.readline()
if s.strip():
    print(int(s))
```

## 02 One string token

**C++17 - complete program**

```
#include <iostream>
#include <string>
using namespace std; int main() {
    string s;
    if (cin >> s) cout << s << '\n';
}
```

WATCH: A token ends at whitespace. For names or sentences containing spaces, use whole-line input (p. 3). This Python example reads the whole small input.

Read one word; print it unchanged.

IN: Kabul

OUT: Kabul

**PYTHON 3 - complete program**

```
import sys
words = sys.stdin.read().split()
if words:
    print(words[0])
```

## 03 Several space-separated integers

**C++17 - complete program**

```
#include <iostream>
using namespace std; int main() {
    long long a, b, c;
    if (!(cin >> a >> b >> c)) return 0;
    cout << a + b + c << '\n';
}
```

WATCH: Both versions accept newlines between the values. Python unpacking requires exactly three tokens. C++ long long still has a finite range: assume the sum fits.

Read exactly three integers; print their sum.

IN: 8 -3 10

OUT: 15

**PYTHON 3 - complete program**

```
import sys
a, b, c = map(int, sys.stdin.read().split())
print(a + b + c)
```

## 04 A pair: integer + word

**C++17 - complete program**

```
#include <iostream>
#include <string>
using namespace std; int main() {
    int id;
    string name;
    if (!(cin >> id >> name)) return 0;
    cout << name << ' ' << id << '\n';
}
```

WATCH: The name is one token by contract. An ID followed by a full name needs line parsing; do not silently discard the rest of the name.

Read an ID and a one-word name; print name then ID.

IN: 17 Farid

OUT: Farid 17

**PYTHON 3 - complete program**

```
import sys
parts = sys.stdin.read().split()
student_id = int(parts[0])
name = parts[1]
print(name, student_id)
```

FAST OUTPUT Use \n instead of endl when a flush is unnecessary. Python print adds a newline; for many small answers collect strings and write "\n".join(out). Interactive tasks need explicit flushing.

MANUAL LOCATOR I/O 1-6 | STL/Python 7-12 | Arrays 13-18 | Greedy/search 19-22 | Graphs 23-30 | DP 31-35 | Math 36-37 | Archetypes 38-44 | Templates 45-47 | Decisions/index 48-50. Pages 7-50 are planned, not yet issued.

PYTHON INPUT LAYERS Use sys.stdin or sys.stdin.buffer consistently; mixing them can interact with buffering. read().split() loses line and case boundaries.

# Choose the correct test-case loop

The stopping condition is part of the input format. A known fixed count, a count in the first token, EOF, and a sentinel are different contracts. Samples below use small sums so the parsing structure is easy to verify.

## 05 Fixed case count specified in the statement

**C++17 - complete program**

```
#include <iostream>
using namespace std; int main() {
    for (int tc = 0; tc < 2; ++tc) {
        long long a, b;
        if (!(cin >> a >> b)) return 0;
        cout << a + b << '\n';
    }
}
```

WATCH: The literal 2 comes from this illustrative specification. Never invent a case count for a paper that only describes one instance.

Exactly TWO cases, each a pair; print each pair sum. No test-count token.

IN: 2 5 / 9 1

OUT: 7 / 10

**PYTHON 3 - complete program**

```
import sys
it = iter(map(int, sys.stdin.read().split()))
for _ in range(2):
    a, b = next(it), next(it)
    print(a + b)
```

## 06 Leading test count, variable-sized cases

**C++17 - complete program**

```
#include <iostream>
using namespace std; int main() {
    int T;
    if (!(cin >> T)) return 0;
    for (int tc = 0; tc < T; ++tc) {
        int n; cin >> n;
        long long sum = 0, x;
        for (int i = 0; i < n; ++i) {
            cin >> x; sum += x;
        }
        cout << sum << '\n';
    }
}
```

Read T; each case has n followed by n integers. Print each sum.

IN: 2 / 3 / 4 5 6 / 0

OUT: 15 / 0

**PYTHON 3 - complete program**

```
import sys
it = iter(map(int, sys.stdin.read().split()))
T = next(it)
for _ in range(T):
    n = next(it)
    total = sum(next(it) for _ in range(n))
    print(total)
```

WATCH: Reset per-case variables inside the loop. Consume n values, not necessarily one physical line. Empty cases may have no data line. Inputs here are complete and valid.

## 07 Read complete records until EOF

**C++17 - complete program**

```
#include <iostream>
using namespace std; int main() {
    long long a, b;
    while (cin >> a >> b) {
        cout << a + b << '\n';
    }
}
```

WATCH: Never use while (!cin.eof()): EOF is detected by a failed read. This Python version uses memory proportional to input; use line streaming if one record per line is guaranteed.

No T: read pairs until input ends; print each sum.

IN: 2 3 / -1 8

OUT: 5 / 7

**PYTHON 3 - complete program**

```
import sys
v = list(map(int, sys.stdin.read().split()))
assert len(v) % 2 == 0
for i in range(0, len(v), 2):
    print(v[i] + v[i + 1])
```

## 08 Read until a sentinel record

**C++17 - complete program**

```
#include <iostream>
using namespace std; int main() {
    long long a, b;
    while (cin >> a >> b) {
        if (a == 0 && b == 0) break;
        cout << a + b << '\n';
    }
}
```

WATCH: Use AND for a 0 0 sentinel. STUDY (2022) uses this termination shape, but its non-sentinel records also have a payload; read that payload only after checking the header.

Print pair sums until BOTH values are zero; sentinel produces no output.

IN: 3 2 / 0 4 / 0 0

OUT: 5 / 4

**PYTHON 3 - complete program**

```
import sys
it = iter(map(int, sys.stdin.read().split()))
for a in it:
    b = next(it)
    if a == 0 and b == 0:
        break
    print(a + b)
```

EOF IS NOT AN EMPTY LINE  C++ getline succeeds on an empty line; Python input() returns "" on a blank line but raises EOFError at EOF. sys.stdin.readline() returns "" only at EOF. Do not stop a line loop merely because the line is blank.

CASE RESET  Clear containers, counters, distance arrays, and answer buffers for every case. Keep a global table only when reuse is intentional. Check the sentinel before allocating or consuming its payload.

# Lines, spaces, blank records, and EOF

Use lines when spaces, empty records, or case boundaries matter. [blank] means an empty line in a sample, not literal input.

## 09 Each line has its own number of integers

**C++17 - complete program**

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std; int main() {
    string line;
    while (getline(cin, line)) {
        istringstream row(line);
        long long x, sum = 0; int n = 0;
        while (row >> x) { ++n; sum += x; }
        cout << n << ' ' << sum << '\n';
    }
}
```

Print count and sum for each line. An empty line is a zero-length record.

IN: 4 8 -1 / [blank] / 5

OUT: 3 11 / 0 0 / 1 5

**PYTHON 3 - complete program**

```
import sys
for line in sys.stdin:
    a = list(map(int, line.split()))
    print(len(a), sum(a))
```

WATCH: Reading all tokens would erase line boundaries. split() accepts repeated whitespace; split(" ") creates empty fields between repeated spaces.

## 10 A whole string, including spaces

**C++17 - complete program**

```
#include <iostream>
#include <string>
using namespace std; int main() {
    string s;
    if (!getline(cin, s)) return 0;
    if (s.size() && s.back() == '\r') s.pop_back();
    cout << '[' << s << ']' << '\n';
}
```

WATCH: Do not use strip() when spaces are data. readline() returns an empty string at EOF; a blank input line is "\n". C++ getline removes \n; the extra check tolerates CRLF input.

Read one line; preserve its leading/trailing spaces. Brackets expose them.

IN:  Kabul team

OUT: [ Kabul team ]

**PYTHON 3 - complete program**

```
import sys
s = sys.stdin.readline()
if s != "":
    s = s[:-1] if s.endswith("\n") else s
    s = s[:-1] if s.endswith("\r") else s
    print("[" + s + "]")
```

## 11 A count followed by exactly that many lines

**C++17 - complete program**

```
#include <iostream>
#include <string>
#include <limits>
using namespace std; int main() {
    int n;
    if (!(cin >> n)) return 0;
    cin.ignore(numeric_limits<streamsize>::max(),
               '\n');
    for (int i = 0; i < n; ++i) {
        string s; getline(cin, s);
        if (s.size()&&s.back()=='\r') s.pop_back();
        cout << s.size() << '\n';
    }
}
```

Print each line length; the blank line is the second record.

IN: 3 / Hello Kabul / [blank] / A B

OUT: 11 / 0 / 3

**PYTHON 3 - complete program**

```
import sys
n = int(sys.stdin.readline())
for _ in range(n):
    s = sys.stdin.readline().rstrip("\r\n")
    print(len(s))
```

WATCH: The count occupies its own line. ignore clears its remainder. getline(cin >> ws, s) would skip meaningful empty lines and leading spaces. Length comparison assumes ASCII text.

## 12 Blank lines delimit cases; EOF closes the last

**C++17 - complete program**

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std; int main() {
    string s; long long sum = 0; bool have = false;
    while (getline(cin, s)) {
        istringstream row(s); long long x;
        if (!(row >> x)) {
            if (have) cout << sum << '\n';
            sum = 0; have = false;
        } else {
            do { sum += x; } while (row >> x);
            have = true;
        }
    }
    if (have) cout << sum << '\n';
}
```

Sum integers in each nonempty group. Output 3, then 7.

IN: 1 2 / [blank] / 3 / 4

OUT: 3 / 7

**PYTHON 3 - complete program**

```
import sys
total, have = 0, False
for line in sys.stdin:
    if line.strip():
        total += sum(map(int, line.split()))
        have = True
    else:
        if have: print(total)
        total, have = 0, False
if have: print(total)
```

WATCH: Whitespace-only lines are delimiters here, not records. Ignore repeated separators; flush once at EOF. OVRSPED (2022) uses blank case boundaries but also declares T.

# Delimiters, records, and exact output

Split only the delimiters named by the statement. These are simple contest records, not general quoted CSV. Preserve source order unless sorting is requested. Headers, prompts, debug text, and extra blank lines are output errors.

## 13 Comma-separated integers, optional spaces

**C++17 - complete program**

```
#include <iostream>
#include <sstream>
#include <string>
#include <algorithm>
using namespace std; int main() {
    string s;
    if (!getline(cin, s)) return 0;
    replace(s.begin(), s.end(), ',', ' ');
    istringstream row(s); long long x, sum = 0;
    while (row >> x) sum += x;
    cout << sum << '\n';
}
```

Read one CSV line of numbers; print their sum.

IN: 2, 5,-3, 0

OUT: 4

**PYTHON 3 - complete program**

```
import sys
s = sys.stdin.readline().strip()
a = [int(x) for x in s.split(",")] if s else []
print(sum(a))
```

WATCH: Valid simple numeric CSV only: no quoted fields or missing values. Replacing commas also tolerates malformed repeated commas in C++; do not mistake this for CSV validation.

## 14 name:value records and fixed decimal output

**C++17 - complete program**

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std; int main() {
    string field;
    cout << fixed << setprecision(2);
    while (cin >> field) {
        auto k = field.find(':');
        string name = field.substr(0, k);
        double value = stod(field.substr(k + 1));
        cout << setw(6) << name << ':'
             << setw(7) << value << '\n';
    }
}
```

Read whitespace-separated records; print right-aligned names and two decimals.

IN: tea:12.5 rice:3

OUT: tea: 12.50 / rice: 3.00

**PYTHON 3 - complete program**

```
import sys
for field in sys.stdin.read().split():
    name, value = field.split(":", 1)
    value = float(value)
    print(f"{name:>6}:{value:7.2f}")
```

WATCH: Fields have one colon and no spaces in the name. Width is a minimum, never truncation. setw applies only to the next item; fixed/precision persist. Binary floating point is not exact decimal money.

## 15 Arrow/colon/comma route record

**C++17 - complete program**

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std; int main() {
    string s;
    if (!(cin >> s)) return 0;
    auto a = s.find("->"), b = s.find(':');
    auto c = s.find(',');
    string from = s.substr(0, a);
    string to = s.substr(a + 2, b - a - 2);
    double km = stod(s.substr(b + 1, c - b - 1));
    double speed = stod(s.substr(c + 1));
    cout << from << ' ' << to << ' ' << fixed
         << setprecision(2) << 60 * km / speed
         << '\n';
}
```

Read a directed route: 5 km at 10 km/h. Print endpoints and 30.00 minutes.

IN: a->b:5,10

OUT: a b 30.00

**PYTHON 3 - complete program**

```
import sys
s = sys.stdin.readline().strip()
route, payload = s.split(":", 1)
source, target = route.split("->", 1)
km, speed = map(float, payload.split(","))
print(source, target, f"{60 * km / speed:.2f}")
```

WATCH: Assume valid punctuation, no internal whitespace, and speed>0. In OVRSPED, a comma marks an edge payload; a query has only a time after the colon. Parse first, then choose the graph algorithm (p. 25).

SPLIT A CSV RECORD For id,name,mark with no quoted commas: C++ getline(stream, field, ',') reads one field; Python fields = line.split(',') retains interior empty fields. Trim numeric fields as allowed; do not trim meaningful name spaces. These are API fragments, not standalone programs.

CONVERSION CHECKS C++ stoi/stoll/stod may throw and can accept a valid numeric prefix. For validation, use the consumed-position argument and verify the entire allowed field. Python int/float reject extra non-whitespace characters. Valid contest inputs rarely need a custom parser.

FORMATTING CHECKS fixed << setprecision(2) means two digits after the point. Without fixed, precision counts significant digits. setw is a minimum width. Use zero fill deliberately and restore the fill character. Match case of Yes/No or true/false exactly; booleans differ between C++ and Python.

ROUNDING Printed decimal precision is not a universal rounding rule. C++ round uses halfway away from zero; Python round uses ties to even, and binary representation can affect decimal ties. For exact decimal rules, parse scaled integers and implement the specified rule.

# Grids and rectangular matrices

R rows and C columns need not be equal. Character input handles the mixed binary rows in Waterfall Streams (2023). For integer cells, parse whole numbers.

## 16 Rectangular character grid: compact or spaced

**C++17 - complete program**

```
#include <iostream>
#include <vector>
#include <string>
using namespace std; int main() {
    int R, C;
    if (!(cin >> R >> C)) return 0;
    vector<string> g(R, string(C, '.'));
    int walls = 0;
    for (int r = 0; r < R; ++r)
        for (int c = 0; c < C; ++c) {
            cin >> g[r][c];
            walls += (g[r][c] == '#');
        }
    cout << walls << '\n';
}
```

Read six single-character cells; store the grid and count # cells.

IN: 2 3 / .#. / # . #

OUT: 3

**PYTHON 3 - complete program**

```
import sys
R, C = map(int, sys.stdin.readline().split())
g = []
for _ in range(R):
    row = "".join(sys.stdin.readline().split())
    assert len(row) == C
    g.append(row)
print(sum(row.count("#") for row in g))
```

WATCH: Each row occupies one line for this Python parser. Whitespace is not a cell. This also reads compact/spaced 0/1 grids; do not use it for multi-digit integers or grids where spaces are meaningful.

## 17 Integer matrix with explicit height and width

**C++17 - complete program**

```
#include <iostream>
#include <vector>
using namespace std; int main() {
    int R, C;
    if (!(cin >> R >> C)) return 0;
    vector<vector<long long>> a(R,
        vector<long long>(C));
    long long sum = 0;
    for (int r = 0; r < R; ++r)
        for (int c = 0; c < C; ++c) {
            cin >> a[r][c]; sum += a[r][c];
        }
    cout << sum << '\n';
}
```

Read R*C integer tokens; store a rectangular matrix and print its sum.

IN: 2 3 / 10 -2 0 / 4 5 6

OUT: 23

**PYTHON 3 - complete program**

```
import sys
data = sys.stdin.buffer.read().split()
it = iter(map(int, data))
R, C = next(it), next(it)
a = [[next(it) for _ in range(C)]
     for _ in range(R)]
print(sum(map(sum, a)))
```

WATCH: R is rows, C is columns. Python [[0]*C]*R aliases the same row; use a comprehension. All values and their sum must fit C++ long long. Tokens can wrap across physical lines.

## 18 CSV matrix with row count; width inferred

**C++17 - complete program**

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
using namespace std; int main() {
    int R; if (!(cin >> R)) return 0;
    string s; getline(cin, s);
    vector<vector<long long>> a;
    for (int r = 0; r < R; ++r) {
        getline(cin, s);
        replace(s.begin(), s.end(), ',', ' ');
        istringstream row(s); long long x;
        vector<long long> v;
        while (row >> x) v.push_back(x);
        if (r && v.size() != a[0].size()) return 1;
        a.push_back(v);
    }
    cout << a.size() << ' ' << a[0].size() << '\n';
}
```

Read CSV rows; confirm equal widths and print row count and column count.

IN: 2 / 0,-1,3 / 4,0,-2

OUT: 2 3

**PYTHON 3 - complete program**

```
import sys
R = int(sys.stdin.readline())
read = sys.stdin.readline
a = [list(map(int, read().split(",")))
     for _ in range(R)]
assert all(len(row) == len(a[0]) for row in a)
print(len(a), len(a[0]))
```

WATCH: Assume R>=1 and nonempty valid rows. Unequal height and width means a rectangle, not ragged rows. Minimum Passes (2025) uses this framing; Sudoku fixes nine rows.

MATRIX CHECKLIST Allocate R rows of C values; loop r<R and c<C. Check bounds before reading neighbors. Zero is still a cell. If only a total is needed, accumulate without storing the matrix.

SUDOKU FRAMING Repeat the CSV row parser nine times and require nine values per row. The supplied 2025 Sudoku sample has no row-count header; the generic example above explicitly does.

# Graphs, trees, indexing, and large input

Convert indexing once at the input boundary. Adjacency lists help traversal; an edge list helps Kruskal (p. 27). Reinitialize both for each case.

## 19 Weighted edge list -> undirected adjacency

**C++17 - complete program**

```
#include <iostream>
#include <vector>
#include <utility>
#include <tuple>
using namespace std; int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; if (!(cin >> n >> m)) return 0;
    vector<vector<pair<int,long long>>> g(n);
    vector<tuple<int,int,long long>> edges;
    for (int i = 0; i < m; ++i) {
        int u, v; long long w; cin >> u >> v >> w;
        --u; --v;
        edges.emplace_back(u, v, w);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    cout << g[0].size() << '\n';
}
```

Read n,m then m triples; convert 1-based endpoints to 0-based. Print degree(0).

IN: 3 2 / 1 2 7 / 2 3 4

OUT: 1

**PYTHON 3 - complete program**

```
import sys
data = sys.stdin.buffer.read().split()
it = iter(map(int, data))
n, m = next(it), next(it)
g = [[] for _ in range(n)]
edges = []
for _ in range(m):
    u, v, w = next(it)-1, next(it)-1, next(it)
    edges.append((u, v, w))
    g[u].append((v, w))
    g[v].append((u, w))
print(len(g[0]))
```

WATCH: Assume n>=1 and valid endpoints. For directed edges add only u->v. For an unweighted pair format, read only u,v and use w=1. ROAD (2022) is already 0-based: do NOT subtract 1 there.

## 20 Tree input: n vertices, exactly n-1 edges

**C++17 - complete program**

```
#include <iostream>
#include <vector>
using namespace std; int main() {
    int n; if (!(cin >> n)) return 0;
    vector<vector<int>> g(n);
    for (int i = 0; i < n - 1; ++i) {
        int u, v; cin >> u >> v;
        g[u].push_back(v); g[v].push_back(u);
    }
    cout << g[0].size() << '\n';
}
```

Read a promised tree with 0-based labels. Print degree(0).

IN: 4 / 0 1 / 1 2 / 1 3

OUT: 1

**PYTHON 3 - complete program**

```
import sys
data = sys.stdin.buffer.read().split()
it = iter(map(int, data))
n = next(it)
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = next(it), next(it)
    g[u].append(v); g[v].append(u)
print(len(g[0]))
```

WATCH: Assume n>=1 and a valid tree. n-1 edges alone does not prove connectedness. A singleton has zero edges; never read a phantom extra edge.

## 21 One adjacency-list line per vertex

**C++17 - complete program**

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
using namespace std; int main() {
    int n; if (!(cin >> n)) return 0;
    string s; getline(cin, s);
    vector<vector<int>> g(n);
    long long entries = 0;
    for (int u = 0; u < n; ++u) {
        getline(cin, s);
        replace(s.begin(), s.end(), ',', ' ');
        istringstream row(s); int v;
        while (row >> v) g[u].push_back(v);
        entries += g[u].size();
    }
    cout << entries << '\n';
}
```

Vertex 0 lists 1; vertex 1 lists 0; vertex 2 has no neighbors. Print total entries.

IN: 3 / 1 / 0 / [blank]

OUT: 2

**PYTHON 3 - complete program**

```
import sys
read = sys.stdin.readline
n = int(read())
g = []
for _ in range(n):
    s = read().strip()
    row = list(map(int, s.split(","))) if s else []
    g.append(row)
print(sum(map(len, g)))
```

WATCH: Blank lines are empty lists here: preserve them. Symmetric undirected adjacency input already contains both directions; do not insert reverse edges again. Empty graph n=0 is readable.

LARGE INPUT C++ fast cin is a good default. Python buffer.read().split() holds the input and token objects; integer conversion adds objects. Stream lines when boundaries are known and memory is tight. Reading costs O(input bytes).

FINAL I/O CHECK Confirm case framing, labels, direction, delimiters, blank-line meaning, row widths, units, and output separators. Check for debug output, repeated edges, and state leaking between cases.